

Date: March 4, 2026

Project Path: D:\SecureElectionSystem\secure_election

1. Executive Summary

The Secure Election System is a Django-based web platform designed to conduct online elections with stronger identity assurance than password-only systems. The project combines voter login, facial authentication, vote encryption, one-person-one-vote enforcement, and intelligent proctoring checks to reduce impersonation and malpractice during remote voting.

The system currently supports election and candidate administration, voter authentication workflows, encrypted vote payload storage, proctoring violation logs, and administrative result visualization/export. It is in a functional development/pre-production stage and requires additional hardening for production deployment.

2. Problem Statement

Traditional online voting workflows suffer from several high-risk weaknesses:

- ? Identity spoofing if only username/password is used.
- ? Repeat voting if constraints are weak.
- ? Vote tampering or accidental exposure if votes are stored in plain text.
- ? Malpractice during voting sessions (absence from camera, multiple faces, camera tampering).
- ? Weak auditability and transparency in post-election verification.

This project addresses those weaknesses through layered controls:

- ? Face registration and face authentication before vote casting.
- ? Encrypted candidate payload storage.
- ? DB-level and logic-level one-vote enforcement.
- ? Proctoring event logging and policy-driven warnings/blocking.

3. Objectives

Primary objectives of the project:

- ? Build a secure web-based voting platform.
- ? Ensure each eligible voter can cast exactly one vote.
- ? Integrate face verification as an additional identity factor.
- ? Detect and respond to suspicious behavior during vote session.
- ? Provide transparent result computation for administrators.

Secondary objectives:

- ? Keep architecture modular for future upgrades.
- ? Maintain usability for non-technical voters.
- ? Provide exportable result reporting for administrative review.

4. Technology Stack

4.1 Backend

- ? Python 3.x
- ? Django (web framework)
- ? Django ORM (data modeling and persistence)
- ? SQLite (current database in development)

4.2 Security and Cryptography

- ? `cryptography` (Fernet symmetric encryption)
- ? Secret-derived key generation from environment-configured secret

4.3 Face Authentication and Proctoring

- ? `face\_recognition` Python library for facial encoding and matching
- ? Browser camera capture APIs (`getUserMedia`)
- ? `face-api.js` for client-side face detection checks during voting

4.4 Frontend

- ? Django templates (server-rendered UI)
- ? HTML/CSS/JavaScript
- ? Bootstrap for layout and base styling

4.5 Reporting

- ? ReportLab (already used in admin action for PDF result export)

4.6 Configuration and Environment

- ? `.env`-based runtime configuration
- ? `.gitignore` safeguards for secret and local artifacts

5. System Modules and Responsibilities

5.1 User Module (`users`)

- ? Voter login and logout.
- ? Links Django `User` to voter profile (`Voter`).
- ? Tracks `voter\_id`, registered face image path, and vote status flag.

5.2 Election Module (`elections`)

- ? Manages election metadata: name, schedule, active flag.
- ? Stores vote time limit for proctoring constraints.
- ? Manages candidate records per election.

5.3 Voting Module (`voting`)

- ? Entry gate validates user login and face enrollment.

- ? Voting page requires face authentication session state.
- ? Prevents duplicate votes via `has\_voted` + vote existence check.
- ? Uses transaction lock to reduce race-condition double-voting.
- ? Stores both candidate FK and encrypted candidate payload.
- ? Computes and displays results (with decryption path).

5.4 Proctoring Module (`proctoring`)

- ? Face registration endpoint for one-time enrollment.
- ? Face authentication endpoint before voting.
- ? Violation logging endpoint for real-time proctor events.
- ? Rule model supports configurable warning thresholds.

6. Core Features in Detail

6.1 Multi-step Voter Verification

- ? Standard credential authentication (username/password).
- ? Mandatory face registration before election participation.
- ? Live face authentication prior to entering vote page.
- ? Session flag (`face\_verified`) used to gate voting UI.

6.2 One-Person-One-Vote Enforcement

- ? Application-level checks:
 - ? `voter.has\_voted`
 - ? existing `Vote` record for voter
- ? Database-level relationship:
 - ? one-to-one mapping between voter and vote record
- ? Transaction-level lock (`select\_for\_update`) during vote write flow.

6.3 Vote Encryption/Decryption

- ? Candidate ID is encrypted using Fernet before storage in `encrypted\_candidate`.
- ? Decryption performed for result aggregation where required.
- ? Encryption key derived from `VOTE\_ENCRYPTION\_SECRET` in environment.
- ? Existing historical records can be backfilled through migration logic.

6.4 Intelligent Proctoring

Active violation types:

- ? `NO\_FACE`
- ? `MULTIPLE\_FACES`
- ? `CAMERA\_OFF`
- ? `FACE\_CHANGED`

? `TIME\_EXCEEDED`

Enforcement behavior:

? Immediate block: `MULTIPLE\_FACES`, `FACE\_CHANGED`

? Warning then block: `NO\_FACE`, `CAMERA\_OFF`, `TIME\_EXCEEDED` using configurable `max\_warnings`

6.5 Administrative Controls

? Django admin for election/candidate/vote/proctoring models.

? Decrypted result visualization in admin list context.

? PDF export action for result snapshots.

6.6 UI/UX Work

? Improved dark-theme compatibility for admin custom result panel.

? Fixed proctoring panel behavior and JS reliability on vote page.

7. Security Posture (Current)

Strengths

? Multi-factor-like workflow (credentials + face verification).

? Encrypted vote payload support.

? Duplicate voting defenses across logic and model constraints.

? Proctoring audit trail through log model.

? Secrets moved to `.env` workflow.

Known Weaknesses / Risk Areas

? Client-side proctoring signals can be evaded by advanced attackers.

? Development defaults may remain if `.env` is misconfigured.

? SQLite is not ideal for production concurrency/scale/audit robustness.

? Limited automated testing currently.

? No advanced anti-automation throttling/rate limiting yet.

8. Difficulties Encountered and How They Were Addressed

8.1 Data Model vs View Mismatch

Issue:

? Result logic expected an encrypted field that did not exist in `Vote` model.

Resolution:

? Added `encrypted\_candidate` field and migration.

? Backfill logic added for old records.

8.2 Hard-coded Encryption Secret

Issue:

? Secret in code is insecure and non-portable.

Resolution:

? Key material now derived from environment-backed secret.

? Added `.env` and `.env.example` workflow.

8.3 Proctoring Endpoint Security

Issue:

? Event logging endpoint initially exempted from CSRF and weakly validated.

Resolution:

? Enforced authenticated POST with validation against allowed violation types.

8.4 Vote Page JavaScript Reliability

Issues:

? Broken element IDs.

? Inconsistent CSRF handling in fetch requests.

? Incomplete real-time proctoring signal flow.

Resolution:

? Reworked vote page script for stable CSRF, timer handling, camera checks, and face count checks.

8.5 Session Handling Side Effects

Issue:

? Full session flush after vote can remove unrelated state.

Resolution:

? Scoped cleanup to `face\_verified` key only.

8.6 Admin Theme Inconsistency

Issue:

? Custom result block used light backgrounds that clashed with dark theme.

Resolution:

? Updated custom admin template and generated result HTML colors to align with admin variables/dark palette.

9. Current Functional Status

Working end-to-end flow:

? voter login -> face register (if needed) -> face auth -> cast vote -> result view

Operational modules:

? Users, elections, voting, proctoring, admin customizations

Configuration:

? Environment-driven secrets and runtime flags now supported

10. Production Readiness Assessment

Current stage: Development / Pre-production

Mandatory actions before production:

? Set `DEBUG=False`

? Restrict `ALLOWED\_HOSTS`

? Deploy behind HTTPS

? Enable secure cookie/session settings

? Move from SQLite to PostgreSQL

? Add structured logging + monitoring + alerting

? Add comprehensive tests (unit/integration/security)

? Validate face-recognition pipeline in production hardware/network conditions

11. Suggested Future Enhancements

? Liveness detection (anti-photo/video spoofing)

? Multi-camera/device integrity checks

? Strong audit trail integrity (hash chain/signature)

? Role-based admin permissions refinement

? Election lifecycle workflow (draft/open/closed/frozen)

? Enhanced analytics dashboard for proctoring incidents

? Accessibility improvements for diverse voter groups

? Containerized deployment pipeline (Docker + CI/CD)

12. Conclusion

The Secure Election System demonstrates a practical layered-security voting architecture with biometric verification and proctoring hooks. The recent fixes significantly improve correctness, security baseline, and maintainability. The application is suitable for continued testing/pilot usage and can be moved toward production after completing infrastructure hardening, database upgrade, and deeper automated validation.